

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФГАОУ ВО «СЕВАСТОПОЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет информационных технологий  
ВТШ «СПИ»

ОТЧЕТ  
о выполнении лабораторной работы № 9  
Тема: «Библиотека NumPy»  
по дисциплине  
«Программирование на языке Python»  
Вариант № 4

Выполнил:

Студент группы ИИ/б-25-б-о

Заварзин А. В.

Проверил:

Старший преподаватель кафедры ИТиС

Кузнецов С. А.

Севастополь, 2026

## ЦЕЛЬ РАБОТЫ

Изучить библиотеку NumPy языка Python для работы с векторами и матрицами, научиться применять методы обработки векторов и матриц при решении практических задач.

## ПОСТАНОВКА ЗАДАЧИ И ВАРИАНТ ЗАДАНИЯ

Задание №1: Решите задачу в соответствии с выданным вариантом. Решение провести двумя способами: стандартными средствами языка Python и с помощью библиотеки NumPy. Измерить время выполнения с помощью команды `%timeit`. Результаты сравнить.

Для варианта №4: Создать матрицу  $7 \times 7$  из случайных целых чисел. Обнулить в ней все элементы, которые делятся на 3 без остатка. Вывести матрицу до и после изменений

Задание №2: Вычислите определитель, ранг, норму, вектор собственных значений матрицы  $G$  и матрицу, обратную ей:

$$G = \begin{pmatrix} 2 & 3 + \alpha & 4 + \mu & 6 \\ \beta + 1 & \gamma & \nu & 12 \\ 5 & -2 & 16 & \mu + 1 \\ 2 & 7 & 11 & -6 \end{pmatrix}$$

Задание №3.1: Решите систему линейных уравнений согласно варианту.

$$\begin{cases} 3x_1 + x_2 + x_3 + \gamma x_4 = \alpha \\ x_1 - \mu x_2 + \nu x_3 + 4x_4 = \beta \\ -5x_1 - x_3 - 7x_4 = -5 \\ x_1 - 6x_2 + \alpha x_3 + 6x_4 = 3 \end{cases}$$

Задание №3.2: Найдите решение системы линейных алгебраических уравнений методом наименьших квадратов:

$$\begin{cases} 3x_1 + x_2 + x_3 + \gamma x_4 = \alpha \\ x_1 - \mu x_2 + \nu x_3 + 4x_4 = \beta \\ -5x_1 - x_3 - 7x_4 = -5 \end{cases}$$

#### Задание №4:

1. Сгенерируйте массив из  $n = 500 + 10\mu + \nu$  чисел, распределенных по нормальному закону с математическим ожиданием, равным  $\nu$  и дисперсией, равной  $\mu + 2$ .

2. Разбейте интервал значений на  $k$  подынтервалов. Количество подынтервалов выберите по правилу Стерджеса:  $k = 1 + 3.322 \cdot \lg(n)$ . Значение  $k$  округлите в большую сторону. Подсчитайте абсолютные и относительные частоты попадания в каждый интервал.

3. Результаты представьте в виде датафрейма, столбцы которого должны содержать следующую информацию: номер интервала, левый конец интервала, его середина, правый конец, абсолютные и относительные частоты попадания в интервал.

4. Подсчитайте накопленную сумму частот (функция `cumsum()`), среднее значение элементов массива, медиану, выборочную дисперсию (смещенную и несмещенную), среднеквадратическое отклонение и 30 % квантиль. Поясните полученные результаты.

## ХОД РАБОТЫ

Перед началом работы необходимо отметить, что все задания выполнены в интерактивной среде Jupyter.

**Задание 1.** Ниже приведён листинг кода для решения поставленной задачи посредством стандартных средств языка программирования Python.

```
import random

matrix = []

def matrixOutput(matrix):
    for row in range(7):
        for col in range(7):
            print(f"{matrix[row][col]:3d}", end=' ')
        print("")

for r in range(7):
    row = []
    for c in range(7):
        row.append(random.randint(0, 100))
    matrix.append(row)

print("Сгенерирована матрица 7x7:")
matrixOutput(matrix)

for row in range(7):
    for col in range(7):
        if matrix[row][col]%3 == 0:
            matrix[row][col] = 0

print("\nМатрица после обнуления элементов согласно заданию:")
matrixOutput(matrix)
```

Реализованная программа создаёт матрицу в виде списка с 7 вложенными списками, в каждом из которых по 7 элементов типа int, которые

сгенерированы с помощью функции `randint()` из стандартной библиотеки Python. Затем происходит вывод матрицы на экран с помощью пользовательской функции `matrixOutput()`. После происходит обработка матрицы согласно варианту №4, а именно: замена всех чисел, которые делятся нацело на 3, на 0. Результат работы программы представлен на рисунке 9.1.

```

Сгенерирована матрица 7x7:
18  63  87   8   0   4  55
76  97  57   3  11  97  54
67  65  34  14  50  84  50
26   3  62  47  77  40  46
81  95  40  98  61  28  72
51  68  58  38  20  75  92
39   0  64   5  22  98  17

Матрица после обнуления элементов согласно заданию:
 0   0   0   8   0   4  55
76  97   0   0  11  97   0
67  65  34  14  50   0  50
26   0  62  47  77  40  46
 0  95  40  98  61  28   0
 0  68  58  38  20   0  92
 0   0  64   5  22  98  17

```

Рисунок 9.1 – Результат работы программы (решение стандартными средствами языка Python)

После решения задачи стандартными средствами языка Python был разработан код для работы с матрицами, который реализован на функциях библиотеки для математических вычислений NumPy. Матрица создана с помощью псевдорандомной генерации с помощью функции `randint` из библиотеки `numpy.random`. Параметрами были заданы границы возможных чисел, а также размерность матрицы (параметр `size`).

После генерации матрицы была произведена обработка матрицы с помощью продвинутой индексации, реализованной для итерируемых классов из

библиотеки NumPy.

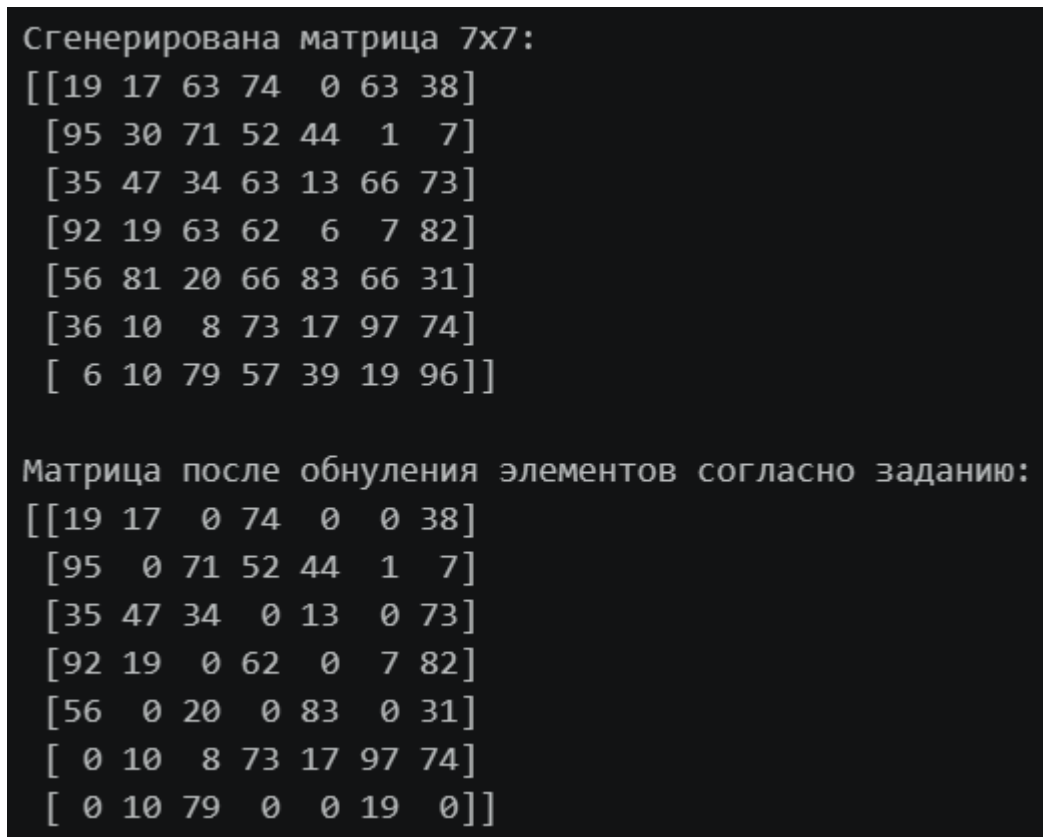
Ниже приведён листинг кода, а также результат его работы (см. рис. 9.2).

```
import numpy as np

matrix = np.random.randint(0, 100, size=(7, 7))
print("Сгенерирована матрица 7x7:")
print(matrix)

matrix[matrix % 3 == 0] = 0 # используем продвинутую индексацию numpy

print("\nМатрица после обнуления элементов согласно заданию:")
print(matrix)
```



```
Сгенерирована матрица 7x7:
[[19 17 63 74  0 63 38]
 [95 30 71 52 44  1  7]
 [35 47 34 63 13 66 73]
 [92 19 63 62  6  7 82]
 [56 81 20 66 83 66 31]
 [36 10  8 73 17 97 74]
 [ 6 10 79 57 39 19 96]]

Матрица после обнуления элементов согласно заданию:
[[19 17  0 74  0  0 38]
 [95  0 71 52 44  1  7]
 [35 47 34  0 13  0 73]
 [92 19  0 62  0  7 82]
 [56  0 20  0 83  0 31]
 [ 0 10  8 73 17 97 74]
 [ 0 10 79  0  0 19  0]]
```

Рисунок 9.2 – Результат работы программы (решение с помощью средств из библиотеки NumPy)

После реализации двух методов решения задачи были произведены замеры скорости работы каждого метода (см. рис. 9.3 и 9.4). Исходя из замеров можно сделать вывод, что продвинутая индексация из библиотеки NumPy работает

намного быстрее по сравнению с обычным перебором каждого списка и элемента с помощью стандартных циклов Python. Выполнение кода с использованием методов и классов из библиотеки NumPy (за 7 запусков) заняло  $13.1 \mu\text{s} \pm 523 \text{ ns}$ , в то время как работа программы с использованием стандартных средств языка (за 7 запусков) заняла  $22.1 \mu\text{s} \pm 907 \text{ ns}$ .

**22.1  $\mu\text{s} \pm 907 \text{ ns}$  per loop (mean  $\pm$  std. dev. of 7 runs, 100,000 loops each)**

Рисунок 9.3 – Время выполнения программы (решение стандартными средствами языка Python)

**13.1  $\mu\text{s} \pm 523 \text{ ns}$  per loop (mean  $\pm$  std. dev. of 7 runs, 100,000 loops each)**

Рисунок 9.4 – Время выполнения программы (решение с помощью средств из библиотеки NumPy)

**Задание 2.** Ниже представлен листинг кода для решения задачи.

```
import numpy as np

# Значения параметров (также будут использоваться дальше в №№3.1, 3.2, 4)
alpha = np.random.randint(0, 25)
beta  = np.random.randint(0, 25)
gamma = np.random.randint(0, 25)
mu     = np.random.randint(0, 25)
nu     = np.random.randint(0, 25)

print(f"Сгенерированные alpha, beta, gamma, mu, nu:")
print(alpha, beta, gamma, mu, nu)
print("\n")

# Матрица G
G = np.array([
    [2, 3 + alpha, 4 + mu, 6],
    [beta + 1, gamma, nu, 12],
    [5, -2, 16, mu + 1],
    [2, 7, 11, -6]
```

```

], dtype=float)

# Определитель
det = np.linalg.det(G)

# Ранг
rank = np.linalg.matrix_rank(G)

# Нормы
norm_fro = np.linalg.norm(G, 'fro')      # норма Фробениуса
norm_2    = np.linalg.norm(G, 2)         # спектральная норма
norm_1    = np.linalg.norm(G, 1)         # столбцовая норма
norm_inf  = np.linalg.norm(G, np.inf)    # строчная норма

# Собственные значения и векторы
eigvals, eigvecs = np.linalg.eig(G)

# Обратная матрица
if abs(det) > 1e-12:
    G_inv = np.linalg.inv(G)
    check = G @ G_inv
    inv_report = f"Обратная матрица:\n{G_inv}\n\nПроверка G @ G-1 (должна  
быть единичная):\n{check}"
else:
    inv_report = "Матрица вырождена — обратной не существует."

# Итог
print("Матрица G:\n", G)
print(f"\nОпределитель det(G) = {det:.6f}")
print(f"Ранг rank(G)          = {rank}")
print(f"\nНорма Фробениуса      = {norm_fro:.6f}")
print(f"Спектральная норма     = {norm_2:.6f}")
print(f"Норма ||G||1          = {norm_1:.6f}")
print(f"Норма ||G||∞          = {norm_inf:.6f}")
print("\nСобственные значения:\n", eigvals)
print("\nСобственные векторы (по столбцам):\n", eigvecs)
print("\n" + inv_report)

```

Данная программа выполняет генерацию случайных значений параметров  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\mu$  и  $\nu$ , далее строит матрицу  $G$  (с типом данных float). Далее идёт нахождение определителя матрицы (функция `det()` из библиотеки `numpy.linalg`), ранга матрицы (функция `matrix_rank()`), всех видов норм (Фробениуса, спектральной, столбцовой, строчной) и нахождение собственных значений и векторов (функция `eig()`). Далее строится обратная матрица (функция `inv()`) с проверкой на вырожденность матрицы (если матрица вырождена – то обратной не существует). В конце программы происходит вывод всех значений на экран. Результат работы программы отображён на рисунке 9.5.

```

Сгенерированные alpha, beta, gamma, mu, nu:
16 10 2 14 16

Матрица G:
[[ 2. 19. 18.  6.]
 [11.  2. 16. 12.]
 [ 5. -2. 16. 15.]
 [ 2.  7. 11. -6.]]

Определитель det(G) = 27327.000000
Ранг rank(G)         = 4

Норма Фробениуса      = 44.384682
Спектральная норма    = 38.760639
Норма ||G||1         = 61.000000
Норма ||G||∞        = 45.000000

Собственные значения:
[ 31.34819171+0.j -16.28083569+0.j -7.87042256+0.j  6.80306655+0.j]

Собственные векторы (по столбцам):
[[ 0.66785597+0.j  0.61017302+0.j -0.67989386+0.j  0.73803076+0.j]
 [ 0.57320638+0.j -0.40358435+0.j  0.35887407+0.j  0.54093686+0.j]
 [ 0.39713661+0.j -0.38193481+0.j -0.20760645+0.j -0.39728981+0.j]
 [ 0.26016412+0.j  0.56474274+0.j  0.60485808+0.j  0.06970452+0.j]]

Обратная матрица:
[[-2.34932484e-02  1.49632232e-01 -1.15051048e-01 -1.18564058e-02]
 [ 5.98309364e-02  2.54693161e-02 -6.21363487e-02 -4.45713031e-02]
 [-1.59183225e-02 -4.34734878e-02  7.15775607e-02  7.60786036e-02]
 [ 3.27880850e-02 -1.09781535e-04  2.03827716e-02 -8.31412156e-02]]

Проверка G @ G-1 (должна быть единичная):
[[ 1.00000000e+00 -1.05709712e-17  1.80411242e-16  1.38777878e-16]
 [-2.77555756e-17  1.00000000e+00  8.32667268e-17  5.55111512e-17]
 [-6.93889390e-18 -5.61074624e-18  1.00000000e+00 -9.71445147e-17]
 [-4.16333634e-17 -1.02457105e-17  5.55111512e-17  1.00000000e+00]]

```

Рисунок 9.5 – Результат работы программы для задания №2

**Задание 3.1.** Ниже приведён листинг кода для решения задачи.

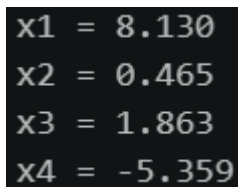
```
left = np.array([
    [3, 1, 1, gamma],
    [1, -mu, nu, 4],
    [-5, 0, -1, -7],
    [1, -6, alpha, 6]
])

right = np.array([alpha, beta, -5, 3]) # матрица из правой части (решения)

result = np.linalg.solve(left, right) # решение

for i in range(1, 5):
    print(f"x{i} = {result[i-1]:.3f}")
```

Описанная программа решает систему линейных уравнений следующим образом: сначала создаётся матрица из коэффициентов (левая часть уравнений без «иксов»), затем создаётся массив из правой части каждого уравнения. После задания двух этих массивов (матриц) вычисляется результат с помощью функции `solve()` (из библиотеки `numpy.linalg`), куда передаются сначала матрица с коэффициентами из левой части, а потом массив из правой части. На выходе получаем массив с неизвестными. Результат работы программы изображён на рисунке 9.6.



```
x1 = 8.130
x2 = 0.465
x3 = 1.863
x4 = -5.359
```

Рисунок 9.6 – Результат работы программы для задания №3.1

### Задание 3.2.      Ниже приведён листинг кода для решения задачи.

```

left = np.array([
    [3, 1, 1, gamma],
    [1, -mu, nu, 4],
    [-5, 0, -1, -7]
])

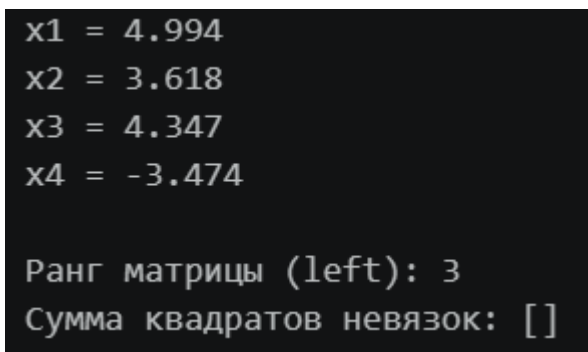
right = np.array([alpha, beta, -5])

# решение, сумма квадратов невязок, ранг, сингулярные числа
result, residuals, rank, s = np.linalg.lstsq(left, right, rcond=None)

for i in range(1, 5):
    print(f"x{i} = {result[i-1]:.3f}")
print(f"\nРанг матрицы (left): {rank}")
print(f"Сумма квадратов невязок: {residuals}")

```

Разработанная программа позволяет решить систему линейных уравнений методом наименьших квадратов. Для этого аналогично предыдущему заданию составляются матрицы (массивы) из левой и правой части уравнений, а затем с помощью функции `lstsq()` находятся: решение, сумма квадратов невязок, ранг и сингулярные числа (функция возвращает кортеж с соответствующими значениями в описанном порядке). Результат работы программы представлен на рисунке 9.7.



```

x1 = 4.994
x2 = 3.618
x3 = 4.347
x4 = -3.474

Ранг матрицы (left): 3
Сумма квадратов невязок: []

```

Рисунок 9.7 – Результат работы программы для задания №3.2

**Задание 4.** Ниже представлен листинг кода для решения задачи.

```

# Задание параметров и генерация массива
n = int(np.ceil(500 + 10 * mu + nu))
print(f"Объем выборки n = {n}")

# Генерация: loc = мат. ожидание ( $\nu$ ), scale = СКО ( $\sqrt{\mu+2}$ )
arr = np.random.normal(loc=nu, scale=np.sqrt(mu + 2), size=n)

# Разбиение на интервалы
k = int(np.ceil(1 + 3.322 * np.log10(n)))
print(f"Количество подынтервалов k = {k}\n")

xmin, xmax = arr.min(), arr.max()           # мин. и макс. значения
edges = np.linspace(xmin, xmax, k + 1)      # k+1 границ для k интервалов
freq, _ = np.histogram(arr, bins=edges)      # абсолютные частоты
rel_freq = freq / n                          # относительные частоты
midpoints = (edges[:-1] + edges[1:]) / 2     # середины интервалов
cum_freq = np.cumsum(freq)                   # накопленные частоты

# Формирование датафрейма (можно было с помощью pandas)
table_list = []
for i in range(k):
    table_list.append({
        'Номер интервала': i + 1,
        'Левый конец': edges[i],
        'Середина': midpoints[i],
        'Правый конец': edges[i + 1],
        'Абсолютная частота': freq[i],
        'Относительная частота': rel_freq[i],
        'Накопленная частота': cum_freq[i]
    })

print("ТАБЛИЦА РАСПРЕДЕЛЕНИЯ:")
print(f"{'№':<3} {'Левый':>10} {'Середина':>10} {'Правый':>10} "
      f"{'Абс.част':>10} {'Отн.част':>10} {'Накопл.':>8}")
print("-" * 67)

for row in table_list:
    print(f"{'row['Номер интервала']':<3} "

```

```

f"{row['Левый конец']:>10.4f} "
f"{row['Середина']:>10.4f} "
f"{row['Правый конец']:>10.4f} "
f"{row['Абсолютная частота']:>10d} "
f"{row['Относительная частота']:>10.4f} "
f"{row['Накопленная частота']:>8d}")

# Расчёт стат. характеристик и вывод
mean_val = np.mean(arr)
median_val = np.median(arr)
var_biased = np.var(arr) # Смещенная дисперсия (ddof=0)
var_unbiased = np.var(arr, ddof=1) # Несмещенная дисперсия (ddof=1)
std_dev = np.std(arr) # СКО (смещенное)
quantile_30 = np.quantile(arr, 0.30)

print("\nСТАТИСТИЧЕСКИЕ ХАРАКТЕРИСТИКИ:")
print(f"Среднее значение (mean): {mean_val:.4f}")
print(f"Медиана (median): {median_val:.4f}")
print(f"Смещенная дисперсия: {var_biased:.4f}")
print(f"Несмещенная дисперсия {var_unbiased:.4f}")
print(f"Среднеквадратическое отклонение: {std_dev:.4f}")
print(f"30% квантиль: {quantile_30:.4f}")

```

Данная программа проходит все этапы, описанные в задании. Сначала генерируется массив (выборка) заданного размера с помощью нормального распределения с математическим ожиданием  $M(X) = \nu$  и дисперсией  $D(X) = \mu + 2$ . Для генерации выборки используется функция `normal` (из библиотеки `numpy.random`). Далее идёт разбиение выборки на  $k$  интервалов, рассчитанных по формуле Стерджеса:

$$k = 1 + 3,322 * \lg(n)$$

После нахождения количества интервалов находятся: минимальное и максимальное значение из массива, границы интервалов, абсолютные и относительные частоты, середины интервалов и накопленные частоты (методы `min()`, `max()`; функции `linspace()`, `histogram()`, `cumsum()`). После формируется датафрейм согласно заданию в котором описаны: номер интервала, левый и

правый конец интервала, середина интервала, абсолютные и относительные частоты попадания в интервал. Далее идёт расчёт основных статистических характеристик, а именно: среднее значение, медиана, дисперсия (смещённая и несмещённая), СКО и 30%-ый квантиль (функции `mean()`, `median()`, `var()` и `std()`, `quantile()`). Результат работы программы изображён на рисунке 9.8.

Объем выборки n = 656  
Количество подынтервалов k = 11

ТАБЛИЦА РАСПРЕДЕЛЕНИЯ:

| №  | Левый   | Середина | Правый  | Абс.част | Отн.част | Накопл. |
|----|---------|----------|---------|----------|----------|---------|
| 1  | 4.2761  | 5.4783   | 6.6804  | 3        | 0.0046   | 3       |
| 2  | 6.6804  | 7.8826   | 9.0848  | 27       | 0.0412   | 30      |
| 3  | 9.0848  | 10.2870  | 11.4892 | 65       | 0.0991   | 95      |
| 4  | 11.4892 | 12.6914  | 13.8936 | 110      | 0.1677   | 205     |
| 5  | 13.8936 | 15.0958  | 16.2980 | 144      | 0.2195   | 349     |
| 6  | 16.2980 | 17.5002  | 18.7024 | 148      | 0.2256   | 497     |
| 7  | 18.7024 | 19.9046  | 21.1068 | 96       | 0.1463   | 593     |
| 8  | 21.1068 | 22.3090  | 23.5112 | 36       | 0.0549   | 629     |
| 9  | 23.5112 | 24.7134  | 25.9156 | 20       | 0.0305   | 649     |
| 10 | 25.9156 | 27.1178  | 28.3200 | 6        | 0.0091   | 655     |
| 11 | 28.3200 | 29.5221  | 30.7243 | 1        | 0.0015   | 656     |

СТАТИСТИЧЕСКИЕ ХАРАКТЕРИСТИКИ:

Среднее значение (`mean`): 15.9507  
Медиана (`median`): 15.9285  
Смещенная дисперсия: 16.5657  
Несмещенная дисперсия: 16.5910  
Среднеквадратическое отклонение: 4.0701  
30% квантиль: 13.7903

Рисунок 9.8 – Результат работы программы для задания №4

## **ВЫВОДЫ ПО РАБОТЕ**

В ходе выполнения работы были приобретены практические навыки работы с библиотекой для тяжёлых математических вычислений NumPy для языка программирования Python. Был проведён сравнительный анализ времени выполнения одних и тех-же операций посредством стандартных функций языка Python, а также с использованием библиотеки NumPy. Получены навыки работы с массивами и матрицами через библиотеку NumPy, а также навыки вычисления различных статистических, вероятностных и математических статистик. Исследованы методы решения линейных уравнений с помощью функций библиотеки NumPy.